

Rappresentazione dei numeri decimali

L'approssimazione

Ci siamo divertiti a convertire i numeri interi, ma come ci dobbiamo comportare con i numeri con la virgola? Soprattutto con quelli periodici o irrazionali, che hanno infinite cifre? Quando lavoriamo con un calcolatore abbiamo una quantità definita di memoria a disposizione, quindi l'unico modo per rappresentare i numeri con virgola è di approssimarli. Può sembrare poco corretto, ma si tratta di approssimazioni così fini che il valore salvato nel computer differisce di pochissimo dal valore “matematicamente corretto”!

Virgola fissa

Il primo modo, molto intuitivo, di convertire i numeri con la virgola è semplicemente stabilire di dedicare un certo numero di bit alla parte intera e un certo numero di bit alla parte decimale, calcolando il bit iniziale per il segno: per esempio, avendo un byte (8 bit – 7 a disposizione perché il primo è il segno, vedi la [codifica MS < https://blue3141.dev/knowledge-base/rappresentazione-dei-numeri-interi/>](https://blue3141.dev/knowledge-base/rappresentazione-dei-numeri-interi/) se non ti ricordi!), decidiamo di tenere 4 bit per la parte intera e 3 per la parte decimale. Quindi la parte intera, insieme al bit di segno, rappresenta i numeri $-(2^5 - 1), 2^5 - 1$ mentre quella decimale $(0, 1 - (\frac{1}{2})^3)$, in questo caso, ma vedremo bene dopo questo discorso. Sono ben pochi numeri...ma ora vediamo come convertire!

Rappresentazione in virgola fissa

Codifica:

- 1) Si mette 0 come MSB se il numero è positivo, 1 se è negativo.
- 2) Si converte in binario la parte intera, assicurandosi che non ecceda il numero di bit dedicati alla parte intera (altrimenti il numero non è rappresentabile).
- 3) Si moltiplica la parte decimale per 2 e si annota la parte intera del risultato. Si prende la parte decimale del risultato e si moltiplica nuovamente per 2, finché non si ottiene parte decimale uguale a 0 o si esauriscono i bit a disposizione per la parte decimale. I risultati vanno trascritti in ordine!
- 4) Si compone il numero: il primo bit è il segno, poi la parte intera, poi la parte decimale.

Decodifica:

- 1) Si mette un meno se il primo bit è 1, poi lo si scarta (anche se è 0).
- 2) Sapendo dove è posizionata la virgola, si converte il numero normalmente: con parte intera di n bit e m bit per la parte decimale, si partirà a moltiplicare da 2^{n-1} , controllando di arrivare a 2^0 alla cifra a sinistra della virgola, poi si prosegue con 2^{-1} , 2^{-2} fino a 2^{-m} .

Esempio 1

Codifichiamo e poi decodifichiamo il numero 6.234 supponendo di avere 1 bit per il segno, 3 bit per la parte intera e 4 bit per la parte decimale.

Cominciamo mettendo 0 come MSB dato che il numero è positivo:

$$6.234_{10} = 0 \dots_2$$

Abbiamo 3 bit per la parte intera: quindi il 6 sarà codificato come 110.

$$6.234_{10} = 0110 \dots_2$$

Ora proseguiamo con la parte decimale: moltiplichiamo per 2 finché non otteniamo parte decimale uguale a 0 (e le cifre rimanenti saranno zeri), oppure finché non finiamo le cifre a disposizione, in questo caso 4.

$$0.234 \cdot 2 = \text{\textcircled{0.468}} \rightarrow 0$$

$$0.468 \cdot 2 = \text{\textcircled{0.936}} \rightarrow 0$$

$$0.936 \cdot 2 = \text{\textcircled{1.872}} \rightarrow 1$$

$$0.872 \cdot 2 = \text{\textcircled{1.774}} \rightarrow 1$$

Abbiamo finito le cifre! Purtroppo dobbiamo tenere quest'approssimazione. Quindi il numero sarà

$$6.234_{10} = 01100011_2$$

Se proviamo a ri-convertirlo in base 10, sapendo che ha 3 bit per la parte intera e 4 per la decimale, cominciamo togliendo la prima cifra (il segno) annotando che è positivo:

$$1100011_2 = + \dots_{10}$$

Poi, sapendo che abbiamo n bit di per la parte intera, dobbiamo iniziare a scrivere le potenze di 2 da $n - 1$, così:

2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
1	1	0	0	0	1	1

La doppia linea indica dov'è la virgola, ossia dopo 2^0 ; come possiamo vedere, torna tutto: 3 bit per la parte intera e 4 per quella decimale. Quindi moltiplichiamo e sommiamo, esattamente come faremmo per convertire un numero binario:

2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
1	1	0	0	0	1	1
4	2	0	0	0	0.125	0.0625

$$4 + 2 + 0 + 0 + 0 + 0.125 + 0.0625 = 6.1875$$

Purtroppo abbiamo un errore di $6.1875 - 6.234 = -0.0465$, ma è inevitabile con così poche cifre!

N.B.: Quale range di numeri possiamo rappresentare con n bit per la parte intera ed m per quella decimale? Prima di tutto riflettiamo su una cosa: considerando solo i numeri interi, possiamo dire di andare a “step” di 1: infatti tra un numero rappresentabile e il successivo rappresentabile c'è una differenza di 1. Non a caso è l'unità minima che possiamo rappresentare con le potenze intere, ossia 2^0 . Quando trasferiamo questo discorso ai numeri decimali, lo step diventa uguale alla potenza più piccola che possiamo avere: in questo caso 2^{-m} . Quindi quando rappresentiamo in virgola fissa, andiamo per step di $\left(\frac{1}{2}\right)^m$, e diventa ovvio che se il numero da rappresentare non è divisibile per questa quantità dovrà essere necessariamente approssimato.

Ora: il numero più grande rappresentabile avrà la parte intera più grande possibile, idem per la decimale. Quindi la parte intera sarà uguale a $2^n - 1$, mentre quella decimale a $1 - \left(\frac{1}{2}\right)^m$ (se non capisci perché, prova a pensare: il numero più grande avrà tutti 1. Quindi, per la parte intera dovremo sommare le potenze di 2 da 2^{n-1} a 2^0 , mentre per la parte decimale da 2^{-1} a 2^{-m} . Serie geometriche *welcome back!*).

Se pensiamo invece al numero più piccolo, si potrebbe erroneamente pensare di prendere la minima unità rappresentabile, ma in verità possiamo andare nei negativi; dato che abbiamo il bit di segno, l'intervallo è simmetrico (rivedi la rappresentazione

MS < <https://blue3141.dev/knowledge-base/rappresentazione-dei-numeri-interi/>> se

non lo ricordi), e il numero più piccolo non è nient'altro che l'opposto del più grande. Comodo!

Quindi se abbiamo n bit per la parte intera ed m per quella decimale possiamo rappresentare i numeri decimali, a passi di $1 - 2^{-m}$, in quest'intervallo:

$$(-(2^n - 1) \cdot (1 - 2^{-m}), +(2^n - 1) \cdot (1 - 2^{-m}))$$

Se invece ci chiediamo quanti numeri diversi possiamo rappresentare, con $n + m$ bit rappresentiamo $2^{n+m} - 1$ informazioni diverse; con il bit di segno diventano il doppio (anche numeri negativi), ma dobbiamo toglierne uno perché c'è lo 0 che negato è ancora 0: quindi $2(2^{n+m} - 1) - 1 = 2^{n+m+1} - 3$.

Virgola mobile (floating point)

La rappresentazione in virgola fissa non è efficiente se fissiamo i bit per parte intera e decimale; per questo si è giunti alla rappresentazione in virgola mobile, che permette come dice il nome stesso di “spostare” la virgola dove serve. Ora vediamo come funziona, può essere complicato all'inizio, ma è molto più versatile della rappresentazione in virgola fissa!

Partiamo da un concetto che dovremmo più o meno tutti avere (se non l'abbiamo scordato dalle medie!). Parlo della notazione scientifica: scrivere un numero in notazione scientifica significa scriverlo nella forma

$$n.m_1m_2m_3\dots \cdot 10^p$$

Ossia un numero con una sola cifra intera n , diversa da 0, seguito da cifre decimali $m_1, m_2, \dots$ e moltiplicato per 10^p . Per esempio il numero 30601 sarebbe

$$30601 = 3.0601 \cdot 10^4$$

Il numero non è certo cambiato, è solo la rappresentazione ad essere diversa. Probabilmente molti di voi hanno fatto fisica o chimica, dove questa notazione è fondamentale per capire l'ordine di grandezza di un numero, per fare i calcoli più velocemente o semplicemente per rappresentare un numero altrimenti enorme o piccolissimo (di costanti del genere ne abbiamo tantissime, no? Quella di Avogadro, di Planck, la costante di gravitazione universale, per citarne qualcuna). Assumerò quindi che abbiamo abbastanza dimestichezza con questa rappresentazione.

Proviamo ad applicare questo concetto ai numeri in base 2. Sarebbero dei numeri di questo tipo :

$$n.m_1m_2m_3 \dots \cdot 2^p$$

(Ho messo 2 come base della potenza 2^p ; so che è scritto in base 10, perché se lo scrivessi in binario sarebbe 10^p , ma pensavo che avrebbe fatto più confusione che altro. Solo quel 2 sarà in base 10, promesso!) Anche qui n dovrebbe essere diverso da 0. Ma siamo in base 2, se non è 0 è per forza 1! Quindi il numero sarà obbligatoriamente nella forma

$$1.m_1m_2m_3 \dots \cdot 2^p$$

Quindi ci basterebbe indicare quali sono le cifre $m_1, m_2, \dots$ e l'esponente p per rappresentare quel numero, dato che sappiamo che la prima cifra (parte intera) è sicuramente 1. La codifica floating point sfrutta esattamente questo principio: il primo bit per il segno, i successivi 8 per l'esponente e altri 23 per le cifre $m_1, m_2, \dots$ che prendono il nome di mantissa. Il totale è di 32 bit, ossia una word! Per i double (doppia precisione) abbiamo 1 bit per il segno, 11 per l'esponente e 52 per la mantissa. Purtroppo abbiamo codifiche diverse per mantissa ed esponente, ma nulla di complicato!

Rappresentazione in virgola mobile (floating point)

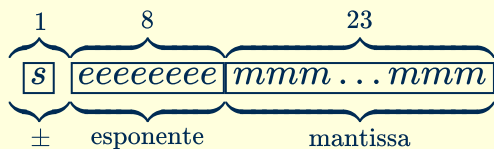
Codifica:

- 1) Si mette 0 come MSB se il numero è positivo, 1 se è negativo.
- 2) Si converte la parte intera in binario, si mette una virgola, e si comincia a convertire la parte decimale come per la rappresentazione in virgola fissa. Si annota dov'è il primo 1 da sinistra (che sia nella parte intera o decimale); si potranno tenere al massimo 23 cifre da quell'uno escluso in poi. Quindi se la parte decimale è rappresentabile esattamente, le altre cifre saranno zeri; altrimenti si converte fino a che non si hanno 23 cifre.
- 3) Si conta di quanti posti bisognerebbe spostare la virgola per portarla a destra del primo 1. Se ci si sposta a sinistra il numero sarà positivo, altrimenti sarà negativo (questo passo non è altro che trovare l'esponente p per ottenere un numero nella forma $1.m_1m_2m_3 \dots \cdot 2^p$).
- 4) Si converte in [eccesso 127](https://blue3141.dev/knowledge-base/rappresentazione-dei-numeri-interi/) < <https://blue3141.dev/knowledge-base/rappresentazione-dei-numeri-interi/> l'esponente e si scrive subito dopo il bit di segno.
- 5) Si scrivono le 23 cifre della mantissa.

Decodifica:

- 1) Si mette un meno se il primo bit è 1, poi lo si scarta (anche se è 0).
- 2) Si prendono le prime 8 cifre e si converte il numero normalmente, togliendo dal risultato 127 (decodifica eccesso 127). Il risultato è l'esponente p .
- 3) Si scrive 1. seguito dalle cifre rimanenti (la mantissa). Si converte normalmente come numero decimale, poi lo si moltiplica per 2^p .

Floating point



Double precision



N.B.: l'esponente non è in eccesso 127 anche per i double. Per capire come codificare l'esponente in base al numero di bit disponibili e si usa il *bias*, che viene calcolato come $b = 2^{e-1} - 1$. La codifica sarà in eccesso b . Per i single precision abbiamo 8 bit, quindi il bias è $b = 2^{8-1} - 1 = 127$ e la codifica sarà in eccesso 127 (codifica: aggiungi 127 e converti, decodifica: converti e toglì 127). Quindi per i double il bias sarà, avendo 11 bit per l'esponente, $b = 2^{11-1} - 1 = 1023$. Avremo quindi una codifica in eccesso 1023.

Esempio 1

Rappresentiamo lo stesso numero di prima, 6.234, in virgola mobile!

Cominciamo mettendo nuovamente 0 come primo bit dato che il numero è positivo:

$$6.234_{10} = 0 \dots$$

Convertiamo la parte intera in decimale:

$$6_{10} = 110_2 \rightarrow \text{\textcircled{110}}$$

Notiamo che abbiamo subito l'1 all'inizio. Quindi, escluso quell'1 (che evidenziamo per comodità) dovremo tenere 23 cifre. 2 sono già prese dalla parte intera, quindi ce ne rimangono 21 per la parte decimale. Cominciamo!

1. $0,234 \cdot 2 = \backslash\text{circled}0.468 \rightarrow 0$
2. $0,468 \cdot 2 = \backslash\text{circled}0.936 \rightarrow 0$
3. $0,936 \cdot 2 = \backslash\text{circled}1.872 \rightarrow 1$
4. $0,872 \cdot 2 = \backslash\text{circled}1.744 \rightarrow 1$
5. $0,744 \cdot 2 = \backslash\text{circled}1.488 \rightarrow 1$
6. $0,488 \cdot 2 = \backslash\text{circled}0.976 \rightarrow 0$
7. $0,976 \cdot 2 = \backslash\text{circled}1.952 \rightarrow 1$
8. $0,952 \cdot 2 = \backslash\text{circled}1.904 \rightarrow 1$
9. $0,904 \cdot 2 = \backslash\text{circled}1.808 \rightarrow 1$
10. $0,808 \cdot 2 = \backslash\text{circled}1.616 \rightarrow 1$
11. $0,616 \cdot 2 = \backslash\text{circled}1.232 \rightarrow 1$
12. $0,232 \cdot 2 = \backslash\text{circled}0.464 \rightarrow 0$
13. $0,464 \cdot 2 = \backslash\text{circled}0.928 \rightarrow 0$
14. $0,928 \cdot 2 = \backslash\text{circled}1.856 \rightarrow 1$
15. $0,856 \cdot 2 = \backslash\text{circled}1.712 \rightarrow 1$
16. $0,712 \cdot 2 = \backslash\text{circled}1.424 \rightarrow 1$
17. $0,424 \cdot 2 = \backslash\text{circled}0.848 \rightarrow 0$
18. $0,848 \cdot 2 = \backslash\text{circled}1.696 \rightarrow 1$
19. $0,696 \cdot 2 = \backslash\text{circled}1.392 \rightarrow 1$
20. $0,392 \cdot 2 = \backslash\text{circled}0.784 \rightarrow 0$
21. $0,784 \cdot 2 = \backslash\text{circled}1.568 \rightarrow 1$

(Di nuovo, so che non dovrei scegliere numeri a caso, ma ogni volta ci casco. Ringrazio Excel di esistere) In esame non succederà di dover convertire un numero così ostico! Ora possiamo scrivere il numero in base 2 con la virgola:

$\backslash\text{circled}110.001110111110011101101$

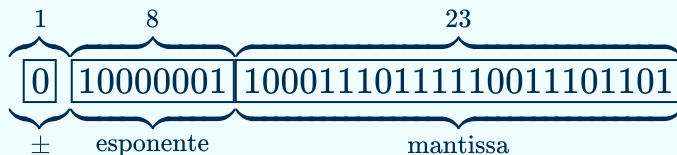
Dobbiamo spostare la virgola a sinistra di due posti per averla a destra del primo 1, quindi il numero sarà moltiplicato per 2^2 .

$\backslash\text{circled}1.10001110111110011101101 \cdot 2^2$

Le cifre decimali (contiamo che siano 23!) le “mettiamo da parte”, sono la nostra mantissa. Per quanto riguarda l'esponente dobbiamo convertirlo in eccesso 127, quindi convertiamo in binario il numero $p + 127$:

$$2 + 127 = 129_{10} = 10000001_2$$

Otto cifre, ottimo! Se ce ne fossero di meno (per l'esponente negativo -126, per esempio, dovremmo codificare $-126 + 127 = 1$, letteralmente 1 anche in base 2) metteremmo a destra degli zeri; se ce ne fossero di più, il numero non è rappresentabile con questa codifica (potremmo provare con un double se l'esponente ha al massimo 11 cifre in binario!). Ed ora mettiamo insieme tutto, bit di segno, esponente e mantissa!



$$6.234_{10} = 01000000110001110111110011101101_{\text{FP}^*}$$

*Una piccola nota. Ora non abbiamo considerato che abbiamo semplicemente troncato il numero: per esempio, per rappresentare 1.919 con due cifre decimali, non tronciamo certo a 1.91, bensì arrotondiamo a 1.92. Con il numero in binario dovremmo fare esattamente la stessa cosa: dopo l'ultima cifra decimale dovremmo convertirne ancora una per capire se arrotondare aggiungendo 1 all'ultima cifra. In questo caso, convertendone ancora una otterremmo un 1, e quindi dovremmo arrotondare l'ultima cifra della parte decimale aggiungendo un'unità. Sarebbe una cosa di questo genere:

FP*	001110111110011101101	+
	000000000000000000001	=
FP	001110111110011101110	

E il numero in codifica floating point sarebbe quindi (metto sotto quello precedente per riferimento)

$$6.234_{10} = 01000000110001110111110011101110_{\text{FP}}$$

$$6.234_{10} = 01000000110001110111110011101101_{\text{FP}^*}$$

La differenza è piccola, ma riconvertendo in decimale il primo (arrotondato) dà 6.23400020599365234375, mentre il secondo (troncato) è 6.233999729156494140625!

Non credo proprio ci verrà chiesto di arrotondare, anche perché se ci chiedessero

davvero di farlo significa che dovremmo fare un sacco di calcoli, e la minicalcolatrice disponibile negli esami online è terribile da usare! Se proprio dovesse capitare sarà per una parte decimale semplice che entra in un loop (per esempio 0.4, provateci! Viene codificato in 0110 periodico).

Esempio 2

Convertiamo un numero più semplice da floating point a decimale. Eccolo qui:

11000001011010000000000000000000_{FP}

Il primo bit è 1, quindi è sicuramente negativo. Lo annotiamo e lo togliamo!

— ...

Prendiamo i successivi 8 bit che codificano l'esponente in eccesso 127:

10000010_{E127} = 130 − 127 = 3

Quindi avremo un numero

— ... · 2³

Ottimo! Ora decodifichiamo la mantissa, antepoendo 1. (che come ricordiamo è implicito):

1.110100000000000000000000

2 ⁰	2 ^{−1}	2 ^{−2}	2 ^{−3}	2 ^{−4}	2 ^{−5}	...
1	1	1	0	1	0	...
1	0.5	0.25	0	0.0625	0	...

Gli altri zeri non li scriviamo, tanto non contribuiscono alla somma. Quindi avremo

1 + 0.5 + 0.25 + 0 + 0.0625 = 1.8125

Questa è la decodifica della mantissa! Scriviamola insieme a ciò che abbiamo ricavato prima e svolgiamo il calcolo:

−1.8125 · 2³ = 14.5

E abbiamo così trovato il nostro numero, 14.5!

E anche qui abbiamo finito! Il prossimo argomento sarà la codifica dei caratteri 😊